Практическая работа. Установка приложений и обновление системы

Ст. преподаватель кафедры ВС
Ревун Артем Леонидович

Задание 1: базовые запросы rpm (1)

Цель: закрепление навыков работы с низкоуровневым менеджером пакетов.

Запрос информации об установленных пакетах:

• Проверьте, установлен ли пакет bash:

```
$ rpm -q bash
```

• Выведите список всех установленных пакетов и подсчитайте их количество:

```
$ rpm -qa | wc -l
```

• Найдите все пакеты, имя которых содержит строку "python":

```
$ rpm -qa | grep python
```

Задание 1: базовые запросы rpm (2)

• Отобразите подробную информацию о пакете: версия, описание, зависимости:

```
$ rpm -qi bash
```

• Выведите список файлов, установленных пакетом bash:

```
$ rpm -ql bash
```

Определение владельца файла:

• Узнайте, какому пакету принадлежит файл /bin/ls:

```
$ rpm -qf /bin/ls
```

• Определите зависимости пакета (что он требует для работы):

```
$ rpm -qR bash
```

Задание 2: работа с .rpm-файлами (1)

Цель: научиться анализировать пакеты перед установкой.

Анализ локального .rpm-файла:

• Скачайте любой .rpm-пакет из репозитория .

• Отобразите метаданные пакета без его установки:

```
$ rpm -qpi package.rpm
```

• Выведите список файлов, которые будут установлены этим пакетом:

```
$ rpm -qpl package.rpm
```

Задание 2: работа с .rpm-файлами (2)

• Проверьте зависимости, которые потребует пакет при установке:
```
$ rpm -qpR package.rpm
```
• Отобразите скрипты pre/post-install из пакета: `$ rpm -qp --scripts package.rpm`

• Проверьте цифровую подпись пакета: `$ rpm -K package.rpm`

Вывод: "digests signatures OK" — подпись верна; "NOT OK" — проблема.

Задание 3: базовые операции с apt (1)

Цель: отработка работы с высокоуровневым менеджером зависимостей.

Обновление индексов и установка пакета:

• Обновите индексы доступных пакетов из репозиториев:

```
# apt-get update
```

• Установите пакет с автоматическим подтверждением:

```
# apt-get install -y htop
```

• Проверьте, что пакет установлен:

```
$ rpm -q htop
```

Задание 3: базовые операции с apt (2)

Удаление пакетов:

• Удалите пакет, сохранив конфигурационные файлы:

```
# apt-get remove htop
```

• Удалите зависимости, которые больше не требуются:

```
# apt-get autoremove
```

• Очистите кэш загруженных .rpm-файлов:

```
# apt-get clean
```

Файлы удаляются из /var/cache/apt/archives/

Задание 4: поиск пакетов с помощью apt-cache (1)

Цель: научиться находить пакеты по имени и описанию.

Поиск и анализ пакетов:

• Найдите пакеты, связанные с редактированием разделов диска:

```
$ apt-cache search hyprland
```

• Отобразите подробную информацию о пакете: версия, размер, описание: `$ apt-cache show gparted`

• Выведите зависимости, которые требует пакет:

```
$ apt-cache depends gparted
```

Задание 4: поиск пакетов с помощью apt-cache (2)

• Найдите пакеты, которые зависят от указанного:

```
$ apt-cache rdepends gparted
```

• Отобразите доступные версии пакета и детали зависимостей:

```
$ apt-cache showpkg gparted
```

Полезные опции:

• --names-only — искать только по имени пакета, а не по описанию.

• --full — показать полную информацию (для show).

• --installed — показать только установленные пакеты.

Задание 5: управление репозиториями (1)

Цель: отработка подключения и отключения источников пакетов.

Просмотр и модификация списка репозиториев:

• Отобразите активные репозитории:

```
# apt-repo list
```

• Покажите все доступные репозитории, включая неактивные:

```
# apt-repo list -a
```

• Добавьте тестовый репозиторий (например, sisyphus):

```
# apt-repo add sisyphus
```

Задание 5: управление репозиториями (2)

• Обновите индексы после изменения списка:

```
# apt-get update
```

• Отключите добавленный репозиторий:

```
# apt-repo rm sisyphus
```

• Проверьте конфигурационные файлы репозиториев:

```
$ cat /etc/apt/sources.list
```

```
$ ls /etc/apt/sources.list.d/
```

Примечание:

• apt-repo rm не удаляет файл, а комментирует строку в конфиге.

Задание 6: обновление ядра системы (1)

Цель: безопасное обновление ядра Linux в ОС Альт.

Автоматическое обновление ядра:

• Обновите индексы пакетов: `# apt-get update`

• Выполните «умное» обновление системы:

```
# apt-get dist-upgrade
```

Разрешает установку новых зависимостей, удаление устаревших пакетов.

• Проверьте, установлено ли новое ядро:

```
$ rpm -qa | grep kernel-image
```

Задание 6: обновление ядра системы (2)

• Обновите конфигурацию загрузчика:

```
# update-kernel
```

Добавляет новое ядро в меню GRUB, обновляет initramfs.

• Перезагрузите систему:

```
# reboot
```

• После перезагрузки проверьте версию ядра:

```
$ uname -r
```

Должна совпадать с версией установленного kernel-image.

Задание 6: обновление ядра системы (3)

Очистка старых ядер:

• Удалите устаревшие версии ядра, оставив текущую и одну предыдущую:

```
# remove-old-kernels
```

• Удалите зависимости, ставшие ненужными:

```
# apt-get autoremove
```

• Проверьте свободное место в /boot:

```
$ df -h /boot
```

Задание 7: установка ПО через epm (1)

Цель: автоматизация установки программ, отсутствующих в репозиториях.

Работа с epm play:

• Установите менеджер epm, если он отсутствует:

```
# apt-get install eepm
```

• Отобразите список доступных для установки программ:

```
$ epm play
```

Вывод: список «плейбуков» — скриптов установки популярных программ.

Задание 7: установка ПО через epm (2)

• Установите программу из списка (например, vscode):

```
# epm play vscode
```

Скрипт скачает пакет с официального сайта, проверит подпись, установит, создаст ярлыки.

• Проверьте, что программа установлена и доступна:

```
$ which code
```

```
$ code --version
```

• Обновите установленную через epm play программу:

```
# epm play --update vscode
```

Задание 7: установка ПО через epm (3)

Массовое обновление:

• Обновите все программы, установленные через epm play:

```
# epm play --update all
```

Проверит версии всех установленных через play программ и обновит при наличии новых.

Задание 8: проверка целостности установленных пакетов (1)

Цель: аудит системы на предмет несанкционированных изменений.

Верификация пакетов:

• Проверьте целостность файлов пакета bash:

```
$ rpm -V bash
```

• Выполните полную проверку всех установленных пакетов:

```
# rpm -Va
```

Может занять время на системах с большим количеством пакетов.

Задание 8: проверка целостности установленных пакетов (2)

- Проанализируйте вывод: символы обозначают изменения:
- . — файл не изменён.
- S — изменён размер файла.
- M — изменены права доступа (режим).
- 5 — изменена контрольная сумма MD5 (содержимое).
- D — изменено устройство (майор/минор).
- L — изменена символическая ссылка.
- U — изменён владелец.
- G — изменена группа.
- T — изменено время модификации.

Задание 8: проверка целостности установленных пакетов (3)

• Найдите пакеты с изменёнными конфигурационными файлами:

```
# rpm -Va | grep '^..5' /etc/
```

Анализ результатов:

• Сохраните отчёт о проверке для последующего анализа:

```
# rpm -Va > integrity_check_$(date +%F).txt
```

• Сравните с предыдущим отчётом при наличии:

```
$ diff integrity_check_old.txt
integrity_check_new.txt
```

Задание 9: резервное копирование состояния пакетной системы (1)

Цель: подготовка к быстрому восстановлению системы после сбоя.

Экспорт списка установленных пакетов:

• Сохраните список установленных пакетов в текстовый файл:

```
# rpm -qa --qf '%{NAME}\n' > packages_$(date +%F).txt
```

• Просмотрите первые и последние строки файла:

```
$ head packages_*.txt
```

```
$ tail packages_*.txt
```

• Подсчитайте общее количество пакетов:

```
$ wc -l packages_*.txt
```

Задание 9: резервное копирование состояния пакетной системы (2)

• Подсчитайте общее количество пакетов:

```
$ wc -l packages_*.txt
```

Резервирование конфигураций:

• Создайте архив с конфигурационными файлами системы:

```
# tar -czf /backup/etc_$(date +%F).tar.gz /etc
```

• Сохраните базу данных rpm для восстановления метаданных:

```
# tar -czf /backup/var-lib-rpm_$(date +%F).tar.gz
/var/lib/rpm
```

• Проверьте целостность созданных архивов:

```
$ tar -tzf /backup/etc_*.tar.gz | head
```

Задание 9: резервное копирование состояния пакетной системы (3)

Восстановление (теоретическая часть):

• Запишите команду для восстановления списка пакетов из файла:

```
# cat packages.txt | xargs apt-get install -y
```

• Запишите команду для распаковки конфигураций в тестовый каталог:

```
# tar -xzf etc_backup.tar.gz -C /tmp/restore_test
```

Задание 10: резервное копирование состояния пакетной системы (3)

Восстановление (теоретическая часть):

• Запишите команду для восстановления списка пакетов из файла:

```
# cat packages.txt | xargs apt-get install -y
```

• Запишите команду для распаковки конфигураций в тестовый каталог:

```
# tar -xzf etc_backup.tar.gz -C /tmp/restore_test
```

Шпаргалка (1)

rpm — низкоуровневый менеджер:

- rpm -q package — проверка установки.
- rpm -qa — список всех пакетов.
- rpm -qi package — подробная информация.
- rpm -ql package — список файлов пакета.
- rpm -qf file — владелец файла.
- rpm -qR package — зависимости пакета.
- rpm -qpi file.rpm — метаданные .rpm-файла.
- rpm -qpl file.rpm — список файлов в .rpm.

Шпаргалка (2)

- rpm -qpR file.rpm — зависимости .rpm-файла.
- rpm -K file.rpm — проверка подписи.
- rpm -V package — проверка целостности.
- rpm -Va — проверка всех пакетов.

apt — менеджер зависимостей:

- apt-get update — обновить индексы.
- apt-get install -y package — установить.
- apt-get remove/purge package — удалить (с конфигом/без).
- apt-get autoremove — удалить ненужные зависимости.

Шпаргалка (3)

- apt-get clean — очистить кэш.
- apt-cache search keyword — поиск пакетов.
- apt-cache show package — информация о пакете.
- apt-cache depends/rdepends package — зависимости.

Репозитории:

- apt-repo list/add/rm — управление источниками.
- apt-cdrom add — добавить CD/DVD.
- /etc/apt/sources.list — основной конфиг.
- /etc/apt/sources.list.d/ — дополнительные конфиги.

Шпаргалка (4)

Ядро и epm:

- apt-get dist-upgrade — умное обновление.
- update-kernel — обновить GRUB и initramfs.
- remove-old-kernels — удалить старые ядра.
- epm play list/install/update — стороннее ПО.

Ресурсы и документация (1)

Официальная документация:

- RPM Guide: https://rpm-software-management.github.io/rpm/
- APT Howto: https://wiki.debian.org/AptHowto
- EPM Documentation: https://www.altlinux.org/Epm

ALT Linux Wiki:

- Управление пакетами: https://www.altlinux.org/Пакеты
- Репозитории: https://www.altlinux.org/Репозитории_Альт_Линукс
- Обновление ядра: https://www.altlinux.org/Обновление_ядра

Ресурсы и документация (2)

Справочные страницы:

$ man rpm # работа с RPM-пакетами

$ man apt-get # управление пакетами через apt

$ man apt-cache # поиск и анализ пакетов

$ man apt-repo # управление репозиториями в Альт

$ man update-kernel # обновление конфигурации загрузчика

Ресурсы и документация (3)

Полезные утилиты:

• rpmrebuild — модификация .rpm-пакетов.

• deborphan — поиск осиротевших пакетов (аналог autoremove).

• needrestart — проверка служб, требующих перезапуска после обновления.

• apt-listchanges — показ изменений в пакетах перед обновлением.